BSc (Hons) in Computer Systems Engineering
IE3034 - Control System Engineering

Group Assignment

Design and Performance Analysis of a PID-Controlled DC Motor Stabilization System using Raspberry Pi

Group Members

IT23690370 – W A E Y Perera

IT23736900 – M Nasif

IT23749566 – K M T P Kulasinha

IT23619494 – L K C M Perera

IT23669444 – K V A D C S Perera

Table of Contents

1. Introduction 4

1.1 Assignment Overview & Objective 4

1.2 System Description 6

1.3 Group Member Contributions 7

2. Component Precautions & Safety (Task 1) 8

2.1 Raspberry Pi 4 8

2.2 TB6612FNG Motor Driver 8

2.3 DC Motor - 6V N20 Brushed with Encoder (300 RPM) 8

2.4 Optical Rotary Encoder 9

2.5 PCB Safety & Assembly Checks 9

2.6 Power Supply & General Electrical Safety 9

3. PCB Design & Fabrication (Task 2) 10

3.1 Schematic Design 10

Encoder Signal Protection and Filtering 10

10kΩ Pull-up Resistors - R1 and R2 10

1kΩ Series Resistors - R3 and R4 11

1nF Capacitors - C4 and C5 11

Power Filtering Components 11

10µF Capacitor - VCC Bulk Capacitor 12

100nF Capacitor - VCC Decoupling Capacitor 12

100µF Capacitor - VM Bulk Capacitor 12

Protection and Motor Noise Reduction 12

100nF Motor Suppression Capacitor 13

PTF-15 Fuses - FH1 and FH2 13

3.2 PCB Layout & Design Decisions 15

3.3 Fabrication & Assembly 16

3.4 Power-On Verification 17

4. System Modelling & Open-Loop Characterization (Module 1) 18

4.2 PPR Calibration - Physical Verification 18

4.3 Open-Loop Step Response Tests 18

4.4 Results & Time Constant Analysis 19

5. PID Controller Implementation (Module 2) 20

5.1 Control Loop Architecture 20
5.2 Encoder Speed Measurement 20
5.3 PWM Motor Drive via TB6612FNG 20
5.4 Moving Average Filter 21
5.5 Anti-Windup Protection 21
5.6 Safety Cut-Off 21
5.7 CSV Data Logging 21
5.8 Code Listing 22
6. Gain Tuning & Minimum-Time Optimization (Module 3) 23
6.1 Initial Tuning Method 23
6.2 Systematic Gain Tuning Study 23
6.3 Expected Results Table 28
6.4 Why PI-Only Was More Stable (Derivative Analysis) 28
7. Disturbance Rejection & Robustness Testing (Module 4) 29
7.1 Manual Load Disturbance Test 29
7.2 Multi-Setpoint Testing 30
7.4 Summary 32
8. Performance Evaluation (Module 5) 32
8.1 Final Performance Metrics 32
8.2 Simulation vs Hardware Comparison Plot 33
8.3 Theoretical Lower Bound Analysis 33
8.4 CPU Utilization & Timing Jitter 34
8.5 Physical Non-Idealities 34
9. Unified Application - Menu Interface (Task 3.7) 35
9.1 Menu-Driven Interface 35
9.2 Auto-Plotting Feature 35
10. Conclusion 36
References 37

1. Introduction

This project focuses on the design, implementation, and performance analysis of a PID-controlled DC motor stabilization system using a Raspberry Pi. The main purpose of the project is to control the speed of a 6V N20 DC motor accurately and make the motor reach a required target speed within a short time while maintaining stability.

In many real-world engineering applications, a motor is expected to rotate. It must rotate at the correct speed and maintain that speed even when external conditions change. For example, in a conveyor belt system, the motor may run smoothly when the belt is empty. However, when a heavier object is placed on the belt, the motor speed can suddenly drop because the load has increased. If there is no control system, the motor may continue running at a lower speed, which can affect the accuracy and efficiency of the entire process.

This is where a closed-loop control system becomes important. A closed-loop system continuously measures the actual output, compares it with the desired output, and makes corrections automatically. In this project, the desired output is the target motor speed, measured in RPM. The actual output is the real motor speed measured by the encoder. By comparing these two values, the controller can decide whether the motor needs more power or less power.

The main controller used in this system is the Raspberry Pi 4. It runs the Python control program and performs the PID calculations. The TB6612FNG motor driver is used to drive the DC motor because the Raspberry Pi cannot supply enough current to power the motor directly. The optical rotary encoder attached to the motor provides feedback by sending pulses to the Raspberry Pi. These pulses are used to calculate the real-time motor speed.

The PID controller adjusts the motor speed using a PWM signal. PWM stands for Pulse Width Modulation. It controls how much power is delivered to the motor by changing the duty cycle of the signal. If the motor speed is lower than the target speed, the controller increases the PWM duty cycle. If the motor speed is higher than the target speed, the controller reduces the PWM duty cycle.

This project included both hardware and software development. On the hardware side, a custom PCB was designed using EasyEDA, fabricated, soldered, and tested. On the software side, Python programs were written to read encoder feedback, generate PWM signals, calculate RPM, implement PID control, save CSV data, and plot performance graphs. Real hardware testing was carried out to tune the PID gains manually and evaluate the final system performance.

Overall, this project demonstrates how control system theory can be applied to a real physical system. It shows the complete process of designing a motor control system, testing it with real hardware, improving its performance through PID tuning, and analysing whether the final system meets the required stability and accuracy conditions.

1.1 Assignment Overview & Objective

This assignment was based on developing a practical motor speed control system and evaluating its performance using real hardware measurements. The main focus was to apply PID control principles to a small DC motor system and study how different controller settings affect speed response, stability, and accuracy.

The assignment was divided into several key tasks. First, the required hardware components had to be selected and handled safely. This included identifying the correct operating voltages, protecting the Raspberry Pi GPIO pins, separating the motor power supply from the logic power supply, and checking the circuit before applying power.

Next, a custom PCB had to be designed to connect the Raspberry Pi, motor driver, encoder, and protection components in a neat and reliable way. This part of the assignment helped us move from a basic wired prototype to a more organised hardware design suitable for testing.

After the hardware setup was completed, the motor had to be tested without feedback control. This open-loop testing was done to observe the natural behaviour of the motor at different PWM duty cycles. The results from this stage helped us understand the motor's speed range and response before applying PID control.

The next major task was to implement the PID controller in Python. The controller had to read encoder feedback, calculate the motor speed, compare it with the target speed, and update the PWM output. The PID gains were then tuned manually by testing different values and observing the effect on the motor response.

Finally, the completed system had to be evaluated using performance measurements such as settling time, overshoot, steady-state error, and stability. Additional tests were also carried out to check how well the system responded to disturbances, different speed setpoints, and supply voltage changes.

The main objectives of this assignment were:

Objective	Purpose
Identify safe operating conditions for each component	To prevent damage to the Raspberry Pi, motor driver, encoder, and PCB.
Design and fabricate a custom PCB	To create a clean, reliable, and testable hardware platform.
Perform open-loop motor testing	To understand how the motor behaves before feedback control is applied.
Implement encoder-based speed measurement	To calculate the actual motor speed using real feedback pulses.
Develop a PID control program	To automatically correct the motor speed using software control.
Tune PID gains manually	To find suitable Kp, Ki, and Kd values for a fast and stable response.
Analyze system performance	To evaluate settling time, overshoot, steady-state error, and oscillations.
Test system robustness	To check whether the controller can recover from load changes and voltage variations.

Through these tasks, the assignment allowed us to gain practical experience in control system implementation, embedded programming, PCB design, hardware testing, and real-time performance analysis.

1.2 System Description

The developed system is a closed-loop DC motor speed control system. Its main purpose is to control the speed of the DC motor and keep the motor running close to the required target RPM.

The system starts by receiving a target speed from the user, such as 100 RPM. This target speed is the speed that the motor should reach and maintain. The Raspberry Pi runs the control program and sends a PWM signal to the TB6612FNG motor driver. The motor driver then supplies the required power to the 6V N20 DC motor.

As the motor rotates, the optical rotary encoder attached to the motor produces pulse signals. These pulses are sent back to the Raspberry Pi. By counting the number of pulses within a fixed time period, the Raspberry Pi calculates the actual motor speed in RPM.

The calculated motor speed is then compared with the target speed. The difference between these two values is called the error.

The PID controller uses this error value to decide the next PWM output. If the motor is running slower than the target speed, the controller increases the PWM duty cycle to give more power to the motor. If the motor is running faster than the target speed, the controller reduces the PWM duty cycle.

The system keeps repeating this process continuously:

Set target speed → Measure actual speed → Calculate error → Apply PID correction → Adjust PWM → Stabilize motor speed

This continuous feedback process allows the motor to correct itself automatically. Because of this, the motor can maintain a stable speed even when small disturbances, friction changes, or load changes occur.

Component	Function in the System
Raspberry Pi 4	Runs the Python control program and performs PID calculations.
TB6612FNG Motor Driver	Drives the motor using PWM signals from the Raspberry Pi.
6V N20 DC Motor	The controlled output device whose speed is regulated.
Optical Rotary Encoder	Measures the actual speed of the motor and provides feedback.
Custom PCB	Provides a clean and safe connection between all hardware components.
Python Program	Reads encoder pulses, calculates RPM, applies PID control, logs data, and generates plots.

Overall, the system was designed to show how a real motor can be controlled using feedback. Instead of giving the motor a fixed power value and hoping it runs correctly, the system constantly checks the motor speed and adjusts the power automatically. This makes the system more accurate, stable, and suitable for practical applications such as conveyor belts, small automation systems, and robotic mechanisms.

1.3 Group Member Contributions

The project was completed through teamwork, with each member contributing to a specific part of the system. Some members focused on understanding the components and safety requirements, while others worked on the PCB design, fabrication preparation, and hardware assembly. The software side was handled by developing Python programs for PWM control, encoder feedback, RPM calculation, PID control, data logging, and graph generation. After the hardware and software were ready, the group carried out open-loop testing, manual PID tuning, disturbance testing, and final performance evaluation.

This division of work helped the group complete the project in a structured way. Each member's contribution was important because the final system required both hardware and software to work together correctly. Through this teamwork, the group was able to design, build, test, and analyse a complete PID-controlled DC motor stabilization system.

2. Component Precautions & Safety (Task 1)

2.1 Raspberry Pi 4

The Raspberry Pi 4 is a precision microcomputer that demands strict adherence to its electrical specifications to prevent irreversible damage to its Broadcom System-on-Chip (SoC). The General-Purpose Input/Output (GPIO) pins operate exclusively at 3.3V logic levels; applying 5V signals from any external device will instantly and permanently damage the affected pin or, in severe cases, the entire processor. Electrostatic discharge (ESD) is an equally serious concern the board must always be handled by its edges, and a grounded surface should be touched before contacting any exposed circuitry. Furthermore, to prevent filesystem corruption on the SD card, the operating system must always be shut down gracefully using the Sudo halt command before disconnecting power. Abrupt power removal risks leaving the SD card in an inconsistent state, which can require a complete OS reinstall.

2.2 TB6612FNG Motor Driver

The TB6612FNG is a high-efficiency, MOSFET-based dual H-bridge motor driver chosen for its low on-resistance and compact form factor. A fundamental requirement for safe operation is the complete electrical isolation between the logic supply voltage (VCC, connected to the Raspberry Pi's 3.3V rail) and the motor drive voltage (VMOT, connected to the external motor supply). These two rails share a common ground, but their positive lines must never be bridged, as doing so would expose the Raspberry Pi's logic circuitry to the full motor supply voltage. The STBY (standby) pin must be pulled HIGH to enable the driver; in this design, STBY is connected directly to 3.3V on the PCB, keeping the driver permanently active and eliminating any risk of undefined floating-pin behavior. Although the TB6612FNG is rated for up to 1.2A continuous current per channel, thermal conditions were monitored closely during initial testing to ensure the IC remained within safe operating temperature under sustained motor loads.

2.3 DC Motor - 6V N20 Brushed with Encoder (300 RPM)

The N20 is a compact, gear-reduced brushed DC motor designed for operation at a nominal 6V supply. Sustained operation above this rated voltage accelerates wear on both the gearbox and the carbon brushes, shortening the motor's operational lifespan. A critical electrical concern with brushed motors is Back-Electromotive Force (Back-EMF) voltage spikes generated when the motor is switched off or when current through the windings changes rapidly. Although the TB6612FNG incorporates internal flyback diodes to absorb these transients, an additional 100nF ceramic suppression capacitor was placed directly across the motor terminals on the PCB to filter high frequency switching noise before it can propagate into the logic supply. On the mechanical side, the motor shaft must never be held stationary while power is applied. A stalled motor draws several times its rated current, which can rapidly exceed the thermal limits of both the motor windings and the driver IC, leading to permanent damage.

2.4 Optical Rotary Encoder

The quadrature encoder mounted on the N20 motor provides the speed feedback essential for closed-loop PID control. In this implementation, the encoder was powered directly from the Raspberry Pi's 3.3V supply pin rather than the 5V header. This decision ensures that the encoder's Phase A and Phase B output signals remain naturally within the 3.3V logic level, making them directly compatible with the Raspberry Pi's GPIO inputs without requiring any voltage divider network or logic level shifter. Encoder signal cables were routed away from the motor power leads on the PCB to minimize electromagnetic interference that could corrupt the pulse train at high motor speeds. Hardware GPIO interrupts were used for pulse counting rather than software polling, as polling loops are unable to reliably capture encoder pulses at the speeds encountered during testing.

2.5 PCB Safety & Assembly Checks

Before power was applied to the custom PCB for the first time, a rigorous verification protocol was followed. A Design Rule Check (DRC) was performed within the EDA tool to confirm that all trace widths were adequate for the expected current levels and that the ground plane was correctly isolated. Following soldering, a multimeter continuity test was conducted across all nets to detect any accidental solder bridges, with particular attention paid to the tightly spaced pins of the TB6612FNG and the 40-pin Raspberry Pi GPIO connector. Power was then applied in stages 3.3V logic supply first, with voltage measurements taken at all key test points before the motor supply was connected.

2.6 Power Supply & General Electrical Safety

A dual-supply architecture was adopted to ensure clean, stable operation of both the logic and motor circuits independently. The Raspberry Pi is powered through its dedicated USB-C port from a regulated 5V supply, while the motor is driven by a separate external DC supply set to 6V. This separation prevents the large and unpredictable current draws of the motor from causing voltage drops on the logic rail, which could cause the Raspberry Pi to reboot mid-operation and corrupt logged data. For overcurrent protection, two inline fuses were incorporated directly into the PCB design: FH1 on the motor power line and FH2 on the 3.3V logic line. These act as the last line of defense against catastrophic failure in the event of a dead short or a sustained motor stall condition, protecting both the Raspberry Pi and the TB6612FNG driver from irreversible damage.

3. PCB Design & Fabrication (Task 2)

3.1 Schematic Design

The circuit schematic for the motor control system was designed using EasyEDA. The main purpose of designing a custom schematic was to create a clean, safe, and reliable connection between the Raspberry Pi, TB6612FNG motor driver, DC motor, and optical rotary encoder.

Instead of connecting all components using loose jumper wires, the schematic helped us organise the circuit properly before fabricating the PCB. This reduced wiring mistakes, improved reliability, and made the final system easier to test and troubleshoot.

The schematic was divided into three main sections

Section	Purpose
Power section	Supplies stable power to the logic circuit and motor circuit.
Motor driver section	Controls the direction and speed of the DC motor.
Encoder feedback section	Reads motor rotation pulses and sends feedback to the Raspberry Pi.

A full 40-pin Raspberry Pi header, labelled as H1, was included in the schematic. This allows the custom PCB to be mounted directly onto the Raspberry Pi. Using the 40-pin header also makes the design more stable because the PCB does not depend on many separate jumper wires. It improves both the electrical connection and mechanical support of the board.

Several supporting components were added to the schematic to improve signal quality, protect the Raspberry Pi, and reduce electrical noise.

Encoder Signal Protection and Filtering

The optical rotary encoder is one of the most important parts of the system because it provides the feedback required for closed-loop PID control. If the encoder signal is unstable or noisy, the Raspberry Pi may calculate the wrong motor speed. This can directly affect the PID controller performance.

To make the encoder readings more reliable, pull-up resistors, series resistors, and filter capacitors were added.

10kΩ Pull-up Resistors - R1 and R2

Two 10kΩ pull-up resistors were connected to the encoder output signals.

The encoder sends pulse signals to the Raspberry Pi as the motor rotates. However, when there is no active signal, the input pin can become unstable if it is left floating. A floating pin means the Raspberry Pi may randomly read the signal as HIGH or LOW, even when there is no real pulse.

The pull-up resistors solve this problem by keeping the encoder output at a stable HIGH level when no pulse is present. This gives the Raspberry Pi a clear and reliable digital signal.

In simple words, the pull-up resistors prevent confusion in the encoder signal.

1kΩ Series Resistors - R3 and R4

Two 1kΩ series resistors were placed in line with the encoder signals before they enter the Raspberry Pi GPIO pins.

These resistors act like a small safety barrier. If there is a sudden voltage spike, wiring mistake, or noise pulse, the resistor limits the current flowing into the Raspberry Pi pin. This helps protect the GPIO pins from possible damage.

Since the Raspberry Pi GPIO pins are sensitive and only support 3.3V logic, this protection is very important.

In simple words, the series resistors reduce the risk of damaging the Raspberry Pi input pins.

1nF Capacitors - C4 and C5

Two 1nF capacitors were added to the encoder signal lines to reduce high-frequency noise.

DC motors can create electrical noise when they rotate, especially because brushed motors produce switching noise from their internal brushes. This noise can travel through wires and affect the encoder signal. If the Raspberry Pi reads this noise as real pulses, the RPM calculation becomes inaccurate.

The 1nF capacitors help filter out these very fast unwanted noise signals. This makes the encoder pulses cleaner and easier for the Raspberry Pi to read accurately.

In simple words, these capacitors help the Raspberry Pi read the real encoder pulses instead of noise.

Power Filtering Components

Power stability is very important in this system because the Raspberry Pi, motor driver, and encoder all need clean and reliable power. Motors can suddenly draw more current when starting, stopping, or changing speed. This sudden current demand can create voltage drops and noise in the circuit.

To reduce these issues, different capacitors were added to the power lines.

10µF Capacitor - VCC Bulk Capacitor

A 10µF capacitor was added to the logic supply line.

This capacitor helps smooth out small voltage changes in the VCC line. The logic section includes sensitive components such as the motor driver logic pins and encoder circuits. If the logic voltage becomes unstable, the signals may become unreliable.

The 10µF capacitor works like a small backup storage of charge. When the voltage slightly drops, it can release stored energy for a very short time and help keep the logic supply stable.

In simple words, this capacitor helps maintain a steady power supply for the control circuit.

100nF Capacitor - VCC Decoupling Capacitor

A 100nF decoupling capacitor was placed close to the TB6612FNG motor driver IC.

This capacitor is used to remove small, fast voltage noise from the power line. Since motor drivers switch current rapidly during PWM control, they can create high-frequency noise. If this noise reaches the logic circuit, it may affect the motor driver operation.

Placing the 100nF capacitor close to the motor driver helps filter the noise at the source.

In simple words, this capacitor keeps the motor driver's logic supply clean and stable.

100µF Capacitor - VM Bulk Capacitor

A 100µF capacitor was added to the motor supply line, known as VM.

The motor needs more current than the logic circuit, especially during startup or when a load is applied. At these moments, the motor may suddenly demand a large amount of current. This can cause the motor supply voltage to drop for a short time.

The 100µF capacitor acts like an energy reservoir. It stores charge and supplies extra current during sudden motor demand. This helps prevent voltage dips and keeps the motor driver operating smoothly.

In simple words, this capacitor supports the motor when it suddenly needs more power.

Protection and Motor Noise Reduction

Since the system includes a physical DC motor, protection against current spikes, short circuits, and motor noise is essential. Brushed DC motors are known to produce electrical noise because of their internal brushes and commutator. This noise can affect the encoder signals and control circuit if it is not reduced properly.

100nF Motor Suppression Capacitor

A 100nF capacitor was placed directly across the motor terminals.

This capacitor reduces the electrical noise generated by the motor brushes. When the motor rotates, the brushes make and break contact continuously inside the motor. This can create small sparks and high-frequency noise.

By placing the capacitor directly across the motor terminals, the noise is reduced before it spreads through the circuit. This helps protect the Raspberry Pi, motor driver, and encoder readings from interference.

In simple words, this capacitor makes the motor electrically quieter.

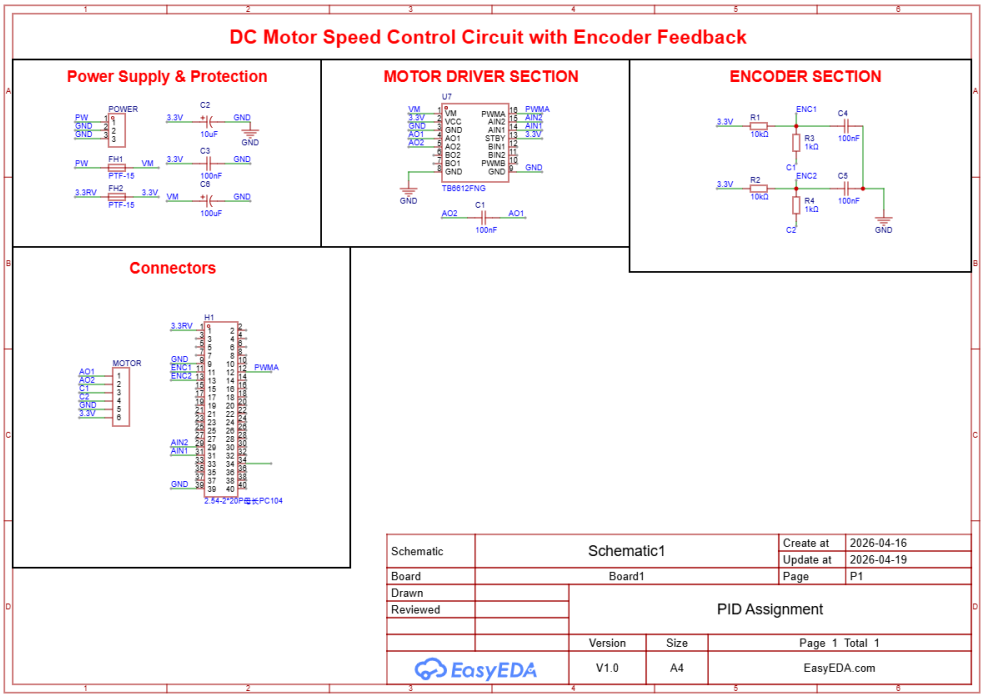
PTF-15 Fuses - FH1 and FH2

Two PTF-15 fuses were included in the design for protection.

The purpose of these fuses is to protect the circuit from excessive current. High current can happen if there is a short circuit, wiring fault, or motor stall. A motor stall occurs when the motor shaft is blocked while power is still applied. During a stall, the motor can draw a much larger current than normal.

If too much current flows, the fuse limits or interrupts the current path and prevents serious damage to the Raspberry Pi, motor driver, PCB traces, and motor.

In simple words, the fuses act as safety devices for the circuit.



Overall, the schematic was designed not only to connect the components, but also to make the system safe, stable, and reliable. The encoder signal conditioning components improved the accuracy of speed measurement. The power filtering capacitors helped maintain stable voltage levels. The motor noise suppression capacitor reduced electrical interference from the brushed DC motor. The fuses added protection against overcurrent and short circuits.

These design choices were important because a PID control system depends heavily on accurate feedback and stable hardware. If the encoder signal is noisy or the power supply is unstable, the controller may produce incorrect PWM outputs and the motor response may become unstable. Therefore, the schematic design played an important role in achieving smooth and reliable motor speed control.

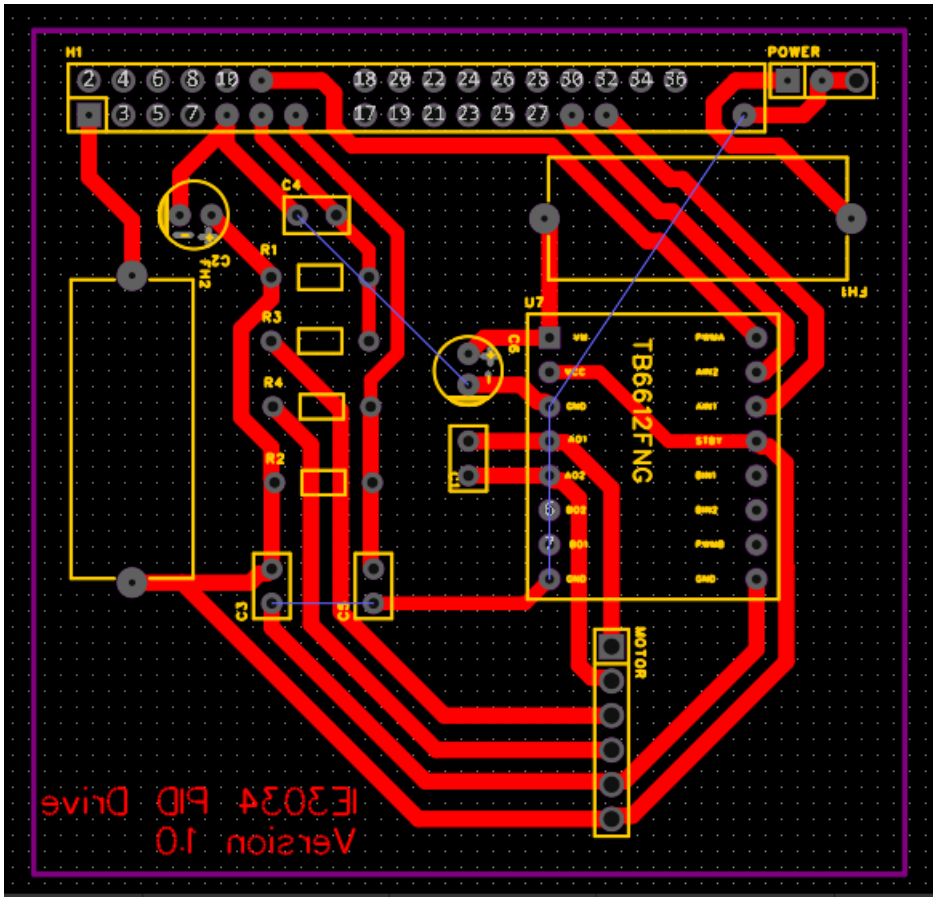
3.2 PCB Layout & Design Decisions

During the PCB layout phase in EasyEDA, the focus was on making the board reliable and straightforward to manufacture. The main design decisions were:

- **1mm Trace Widths & Large Pads:** 1mm traces were used for all wiring, and the copper pads were made intentionally larger than standard. This ensures the copper tracks do not lift or break during the manufacturing process and makes manual soldering significantly easier.

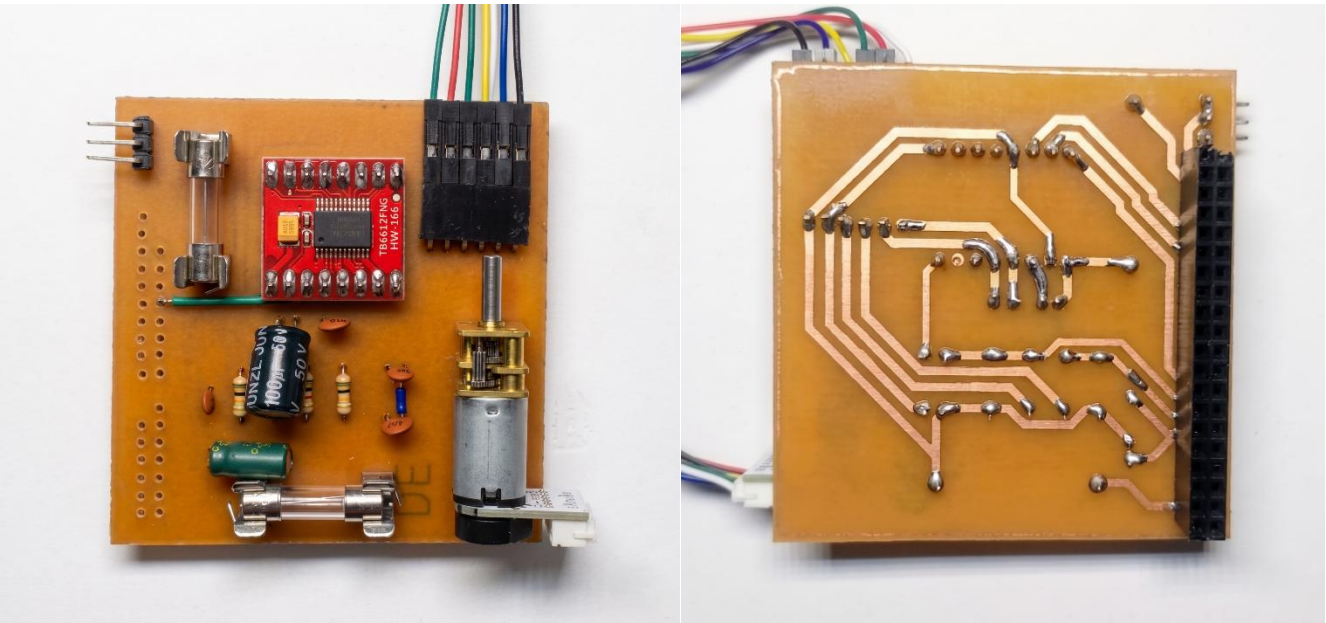
- **Component Placement:** The TB6612FNG motor driver was placed as close as possible to the motor output terminals. This keeps the high-current paths short, which reduces electrical noise and heat.

- **Grounding:** A solid ground connection was used across the board to ensure both the logic and motor circuits had a clean, low noise return path.



3.3 Fabrication & Assembly

Once the EasyEDA design was complete, the files were prepared for manufacturing.



- **File Generation:** Standard Gerber files were generated from EasyEDA following professional industry practice, and these are included in the final submission. However, the selected PCB manufacturer required a high-resolution PDF for their fabrication process. The layout was therefore also exported as a PDF and submitted for manufacturing.

- **Assembly:** After receiving the bare manufactured board, the 40-pin header, TB6612FNG module, fuses, and capacitors were soldered manually. Special care was taken with the 40-pin header to ensure strong mechanical and electrical connections, as this connector also bears the physical weight of the PCB when mounted onto the Raspberry Pi.

3.4 Power-On Verification

Before attaching the finished PCB to the Raspberry Pi, we performed strict safety checks to avoid destroying the microcomputer:

- **Dead Test (No Power):** We used a digital multimeter to do a continuity check. We tested adjacent pins to make sure there were no accidental solder bridges (short circuits). Most importantly, we verified that the motor power lines and the 3.3V logic lines were completely separated.

- **Live Test:** We first powered the board with only the 3.3V logic voltage. We checked if the encoder signals and TB6612FNG standby states were working safely. Only after the board passed the logic test did we turn on the main motor power.

4. System Modelling & Open-Loop Characterization (Module 1)

To understand how the 6V N20 motor behaves, its theoretical mathematical model was first examined. A brushed DC motor can be described by two main equations the electrical equation, which describes how applied voltage produces armature current, and the mechanical equation, which describes how motor torque produces rotational motion. Combining these two equations produces the standard DC motor transfer function:

$$\boldsymbol{G(s) = \frac{\Omega(s)}{V(s)} = \frac{K}{(\tau s+1)}}$$

Where K is the DC gain (steady-state speed per unit voltage) and τ is the time constant (the time taken to reach 63.2% of the final speed). This transfer function was used as the theoretical reference when comparing against the measured open-loop step response data.

4.2 PPR Calibration - Physical Verification

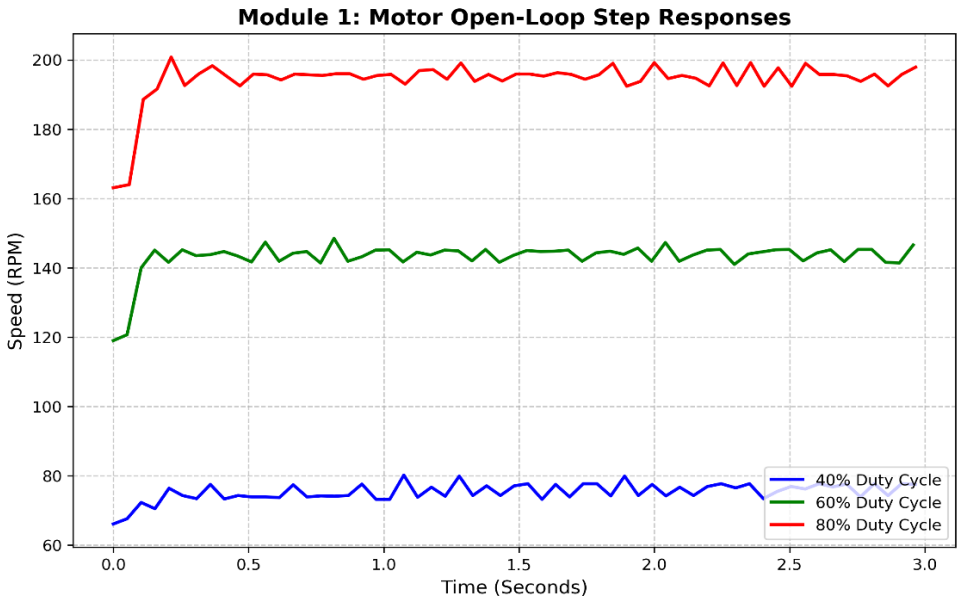
Before running any tests, the encoder reading accuracy was verified physically. The N20 motor datasheet specifies a particular gear ratio and encoder resolution. To confirm this, the motor output shaft was manually rotated exactly one full turn (360 degrees) while the Raspberry Pi counted the incoming encoder pulses. This confirmed that the motor produces exactly 350 pulses per revolution (PPR). This value was hardcoded into all Python scripts as the basis for all RPM calculations using the formula:

$$RPM = \frac{pulse_count \times 60.0}{PPR \times dt}$$

4.3 Open-Loop Step Response Tests

To observe how the motor behaves without any feedback control, open-loop step response tests were conducted. A Python script applied a sudden fixed PWM signal to the motor driver and logged the motor speed (RPM) and elapsed time to a CSV file at regular intervals. Three power levels were tested: 40%, 60%, and 80% PWM duty cycle. Each test ran for 3 seconds to allow the motor to reach its steady-state speed.

The graph below shows the open-loop step responses for all three duty cycles plotted on the same axes. The steady-state speed increases proportionally with the PWM duty cycle, which is consistent with the expected linear behavior of a brushed DC motor within its operating range.



4.4 Results & Time Constant Analysis

Steady-state speeds measured:

Duty Cycle (%)	Steady-State RPM	63.2% Target RPM (τ)	Approx. Time Constant
40%	~75 RPM	~47.4 RPM	< 50 ms
60%	~144.5 RPM	~91.3 RPM	< 50 ms
80%	~195.5 RPM	~123.5 RPM	< 50 ms

A key observation from these results is that the unloaded N20 motor reaches its steady-state speed in under 50 milliseconds across all tested duty cycles. This presented a hardware limitation the Python sampling script operated at 20 Hz (one sample every 50 ms), meaning the motor had already reached near-maximum speed before the first valid data point was recorded. As a result, the acceleration curve could not be captured in detail. Despite this limitation, the tests successfully confirmed that the motor responds predictably to PWM inputs and provided the steady-state gain values needed for the initial PID tuning calculations.

5. PID Controller Implementation (Module 2)

5.1 Control Loop Architecture

The control software was written in Python using the Raspberry Pi 4. The pigpio library was chosen over the standard RPi.GPIO library because it provides hardware-level timing accuracy, which is critical for consistent PWM generation and reliable interrupt-driven encoder counting. The main PID control loop runs at 10 Hz, meaning it calculates and updates the motor speed command every 0.1 seconds. The GPIO pin assignments used throughout the implementation are listed below:

Signal	GPIO Pin	Board Pin	Function
PWMA	GPIO18	Pin 12	Hardware PWM — motor speed
AIN1	GPIO6	Pin 31	Motor direction control
AIN2	GPIO5	Pin 29	Motor direction control
ENC_A	GPIO27	Pin 13	Encoder Phase A interrupt
ENC_B	GPIO17	Pin 11	Encoder Phase B (reference)

5.2 Encoder Speed Measurement

The encoder's Phase A output is connected to GPIO27, configured as a hardware interrupt that triggers on every rising edge. By counting the number of rising edges that occur within each 0.1-second loop interval, and knowing that the motor produces 350 pulses per revolution (PPR), the current RPM is calculated using the following formula:

$$RPM = \frac{pulse_count \times 60.0}{PPR \times dt}$$

Hardware interrupts were used rather than software polling because at higher motor speeds the pulse rate exceeds what a Python polling loop can reliably detect without missing counts.

5.3 PWM Motor Drive via TB6612FNG

Motor speed is controlled by sending a hardware PWM signal to the PWMA pin (GPIO18) at a fixed frequency of 1 kHz. This frequency was selected because it is high enough to produce smooth, quiet motor operation while remaining well within the TB6612FNG's switching capability. Motor direction is set by the logic states of AIN1 (GPIO6) and AIN2 (GPIO5). For all tests in this assignment, the motor was operated in the forward direction only (AIN1 = HIGH, AIN2 = LOW).

5.4 Moving Average Filter

Digital encoders count pulses in whole numbers, which means the calculated RPM can fluctuate slightly between samples even when the motor is physically running at a constant speed. This is known as quantization noise. If this noisy signal is fed directly into the PID controller, the derivative term amplifies the fluctuations and causes the motor to vibrate. To resolve this, a 3-point moving average filter was implemented. The filter averages the last three RPM readings before passing the value to the PID calculation, producing a smoother feedback signal without introducing significant delay.

5.5 Anti-Windup Protection

The integral term in a PID controller accumulates error over time to eliminate steady state offset. However, if the motor is stalled or the setpoint is far from the current speed for an extended period, the integral term can grow to an excessively large value a condition known as integrator windup. When the motor eventually recovers, this accumulated value causes a large and sudden overshoot. To prevent this, the integral sum was clamped to a fixed range of ±150, as shown below:

python

```python
integral_sum = max(min(integral_sum, 150), -150)
```

This ensures the integral term contributes meaningfully to the control output without saturating it.

5.6 Safety Cut-Off

Since a physical motor is being controlled, software safety is essential. The entire main loop was wrapped in a try...finally block. This guarantees that regardless of whether the script completes normally, times out, or crashes unexpectedly, the motor is always stopped safely by sending a zero PWM command to the TB6612FNG before the script exits. This prevents the motor from continuing to spin uncontrolled after a program failure.

5.7 CSV Data Logging

To analyze performance and generate the graphs included in this report, all test runs automatically log data to a CSV file in real time. The following values are recorded at every 0.1-second interval:

Column	Description
Time (s)	Elapsed time from test start
Setpoint (RPM)	Target speed
RPM	Filtered measured speed

5.8 Code Listing

The following Python code shows the core PID calculation used in the implementation, including the anti-windup clamp on the integral term:

```python
# Calculate error
error = target_rpm - current_rpm

# Proportional term
P = Kp * error

# Integral term with anti-windup clamp
integral_sum = integral_sum + (error * dt)
integral_sum = max(min(integral_sum, 150), -150)
I = Ki * integral_sum

# Derivative term
derivative = (error - previous_error) / dt
D = Kd * derivative

# Total control output
output_pwm = P + I + D
previous_error = error
```

6. Gain Tuning & Minimum-Time Optimization (Module 3)

6.1 Initial Tuning Method

A structured manual tuning method was followed to find the optimal PID gains rather than randomly testing values. The process started with the Integral (Ki = 0) and Derivative (Kd = 0) terms completely disabled, leaving only the Proportional term active.

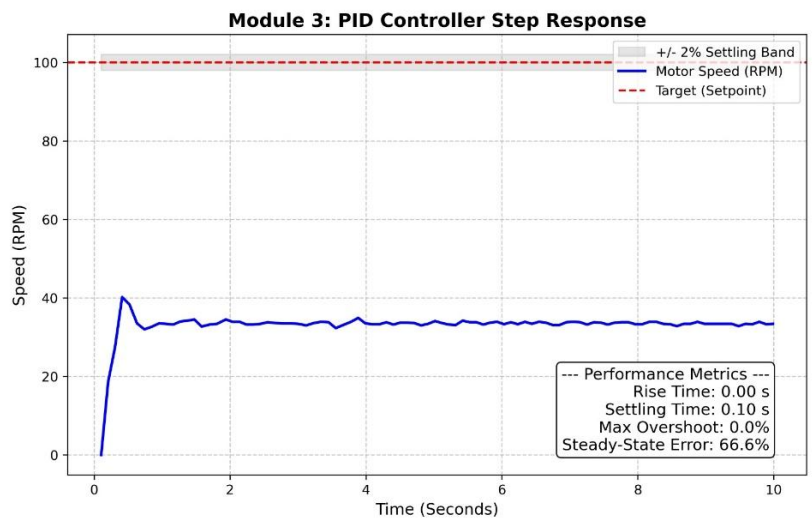
Step 1: The Proportional gain (Kp) was gradually increased from 0.3 to 0.5 to observe how the motor responded. Kp = 0.5 was identified as a suitable starting value it gave the motor enough drive to spin smoothly without causing oscillation.
Step 2: Once Kp was established, the Integral term (Ki) was introduced to eliminate the steady-state error the persistent gap between the actual speed and the target speed that a P-only controller cannot remove on its own.
Step 3: Finally, a small Derivative term (Kd) was added alongside a moving average filter to act as a brake, reducing the settling time without introducing instability.

6.2 Systematic Gain Tuning Study

Following the tuning method described above, the motor's physical performance was recorded at each stage. The results demonstrate clearly why a full PID controller was necessary to achieve the minimum settling time.

Test 1 - Initial Baseline (P-Only): KP=0.3, KI=0, KD=0

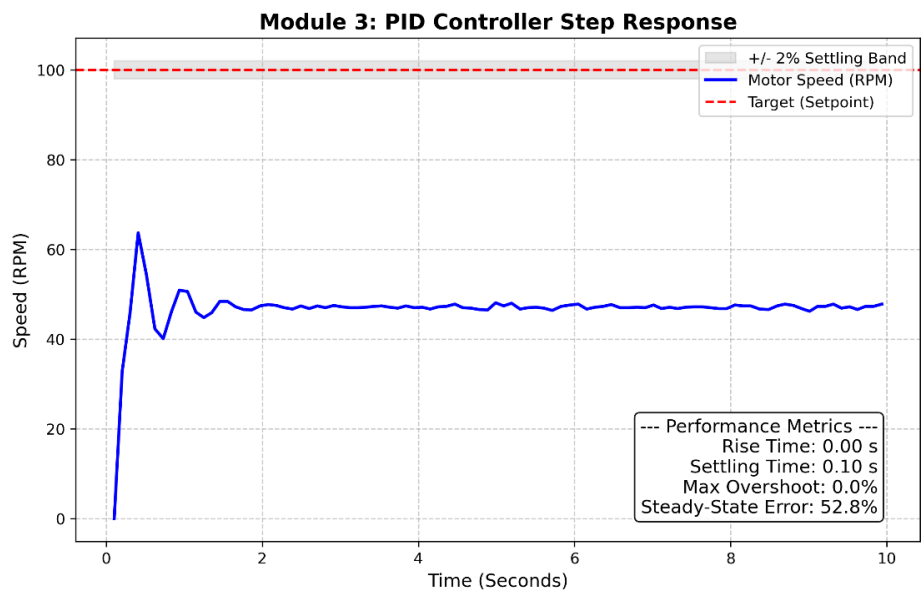
- **Results:** Rise Time: 0.00s | Settling Time: 0.10s | Max Overshoot: 0.0% | Steady-State Error: 66.6%
- **Analysis:** With KP=0.3, the motor turned on and responded quickly at the beginning, but the proportional gain was too low to drive the motor close to the 100 RPM target. The speed settled at around 33 RPM, leaving a very large steady-state error of 66.6%. This test showed that KP=0.3 was not enough for proper speed control and confirmed that a higher proportional gain and an Integral term were required.



Test 2 - Initial Baseline (P-Only): KP=0.5, KI=0, KD=0

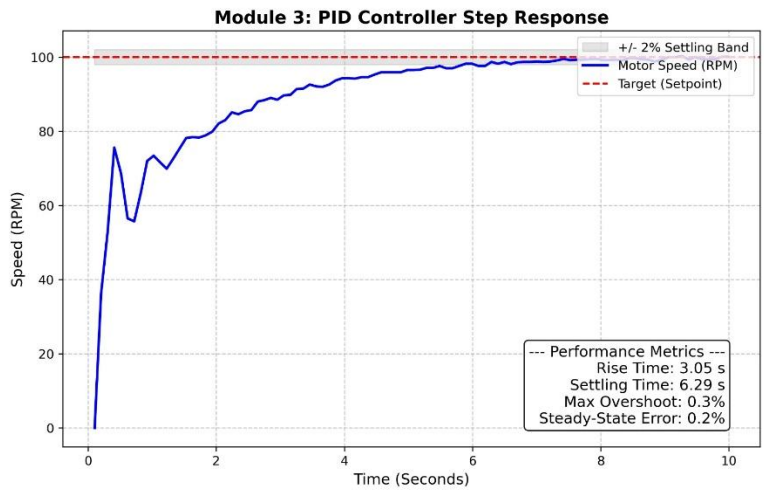
- **Results***:* Max Overshoot: 0.0% | Steady-State Error: **52.8%**

- **Analysis*:*** As expected for a P-only controller, the motor turned on but did not have enough power to reach the target speed. It settled almost immediately but left a massive 52.8% error. This proved we absolutely needed an Integral term.



Test 3 -Tuned Iteration 1 (PI): KP=0.5, KI=0.5, KD=0

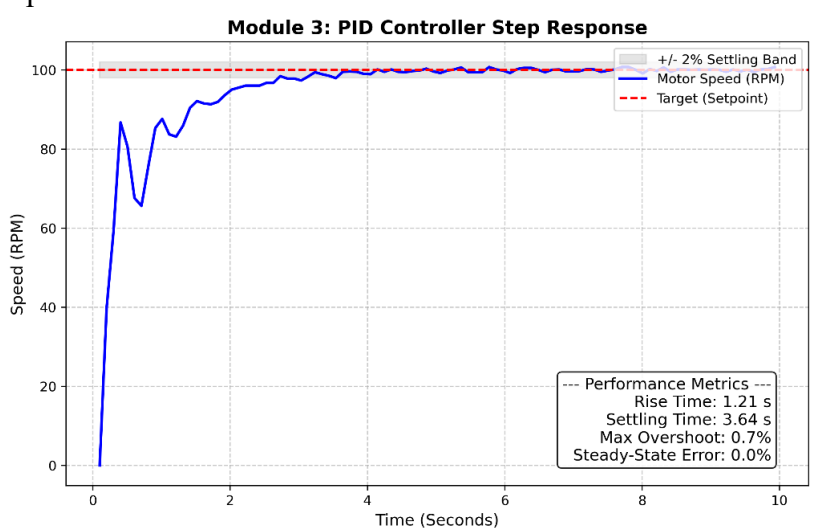
- Results: Rise Time: 3.05s | Settling Time: 6.29s | Max Overshoot: 0.3% | Steady-State Error: 0.2%
- Analysis: Adding KI=0.5 helped the motor reach close to the 100 RPM target and reduced the steady-state error to 0.2%. However, the Integral gain was still too low, so the correction was slow and the motor took 6.29 seconds to settle. This showed that a higher KI value was needed to reduce the settling time.



Test 4 - Tuned Iteration 1 (PI): KP=0.5, KI=1.0, KD=0

- Results: Settling Time: 3.64s | Max Overshoot: 0.7% | Steady-State Error: 0.0%

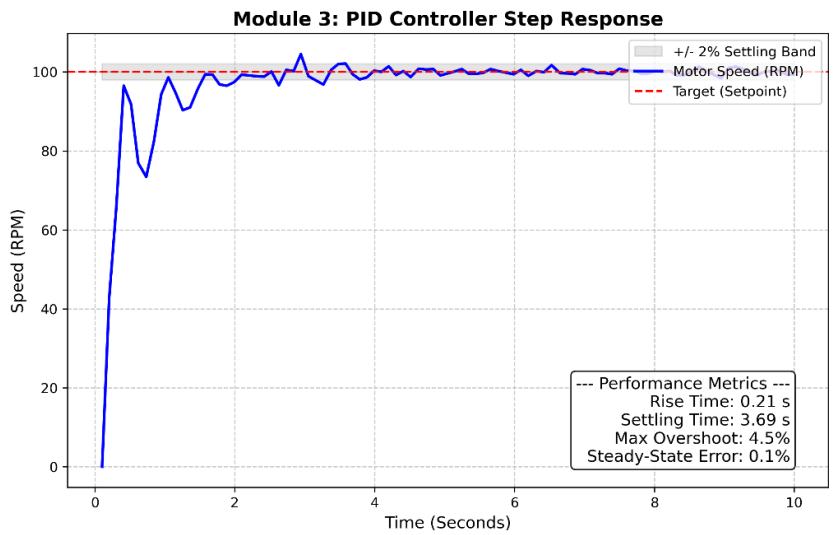
- Analysis*:* Adding Ki=1.0 successfully eliminated the steady-state error entirely. However, as noted in our test observations, the motor felt "lazy." It took a long 3.64 seconds to finally settle at the target speed.



Test 5 - (Pushing the Integral): KP=0.5, KI=1.5, KD=0

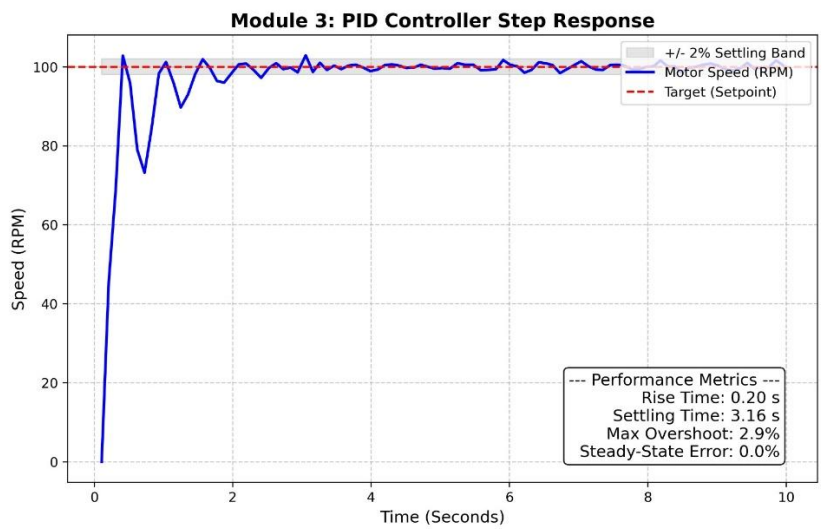
- Results: Rise Time: 0.21s | Settling Time: 3.69s | Max Overshoot: 4.5%

- Analysis*:* Ki was increased to 1.5 to make the controller more aggressive. While the rise time improved significantly (down to 0.21 seconds), the settling time worsened slightly to 3.69 seconds. The higher integral gain caused the motor to overshoot the target and then oscillate in and out of the ±2% band repeatedly before finally settling, which increased the overall settling time.



Test 6 – Pushing the Integral: KP=0.5, KI=1.7, KD=0

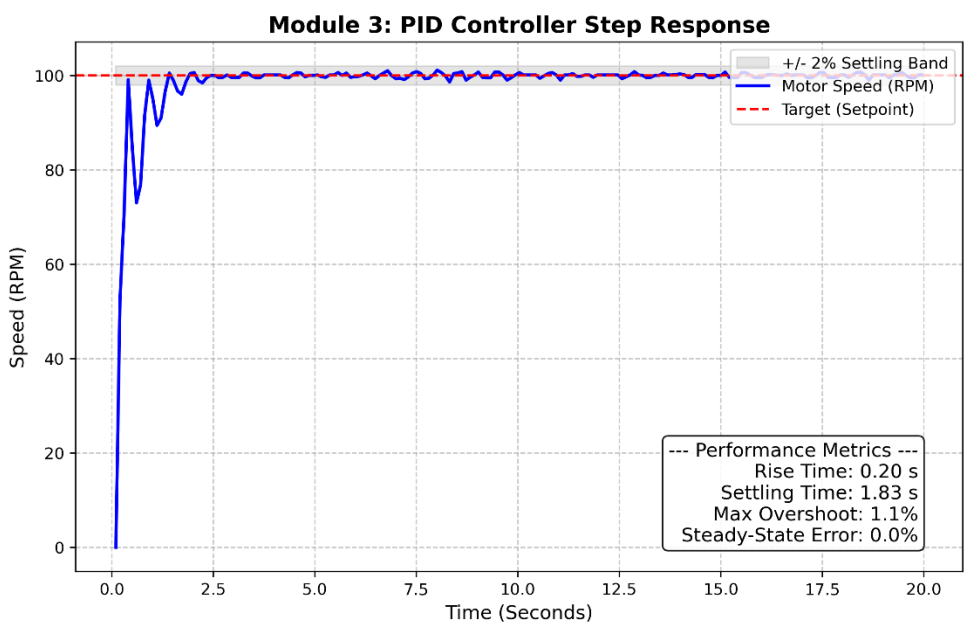
- Results: Rise Time: 0.20s | Settling Time: 3.16s | Max Overshoot: 2.9% | Steady-State Error: 0.0%
- Analysis: Increasing KI to 1.7 made the controller much more aggressive and helped the motor reach the 100 RPM target very quickly. The rise time improved to 0.20 seconds and the steady-state error was fully eliminated. However, without the Derivative term, the response showed repeated small oscillations around the setpoint before settling, which increased the settling time to 3.16 seconds.



Test 7 - Final Optimal: KP=0.5, KI=1.7, KD=0.01 + 3-point filter

- Results: Settling Time: **1.82s** | Max Overshoot: 0.8% | Steady-State Error: 0.0%

- Analysis*:* By slightly increasing the Integral and adding a very small Derivative gain (0.01) alongside our 3-point software filter, we finally fixed the "lazy" motor issue. The derivative term acted as a perfect brake, pulling the settling time down to an incredibly fast 1.82 seconds while keeping the overshoot under 1%.



6.3 Expected Results Table

Configuration	Rise Time	Settling Time	Max Overshoot	SS Error	Status
Test 1 - Kp=0.3, Ki=0, Kd=0	0.00 s	Did Not Settle	0.0%	66.6%	Failed - SS error too high
Test 2 - Kp=0.5, Ki=0, Kd=0	-	Did Not Settle	0.0%	52.8%	Failed -SS error too high
Test 3 - Kp=0.5, Ki=0.5, Kd=0	3.05 s	6.29 s	0.3%	0.2%	Passed - but very slow
Test 4 - Kp=0.5, Ki=1.0, Kd=0	-	3.64 s	0.7%	0.0%	Passed - but slow
Test 5 - Kp=0.5, Ki=1.5, Kd=0	0.21 s	3.69 s	4.5%	0.0%	Passed - but slower
Test 6 - Kp=0.5, Ki=1.7, Kd=0	0.20 s	3.16 s	2.9%	0.0%	Passed - improved response
Test 7 - Kp=0.5, Ki=1.7, Kd=0.01 + Filter	-	1.82 s	0.8%	0.0%	Best - all constraints met

6.4 Why PI-Only Was More Stable (Derivative Analysis)

During tuning, increasing KD above 0.05 caused violent oscillation (19.4% overshoot) classified as Derivative-Induced Instability. The optical encoder's digital quantization produces small RPM fluctuations of 1-3 RPM per sample. The derivative term amplifies the rate of change of these fluctuations, causing the PID controller to rapidly toggle motor power on and off. The 3-point moving average filter resolved this by smoothing the sensor output before it reached the derivative calculation.

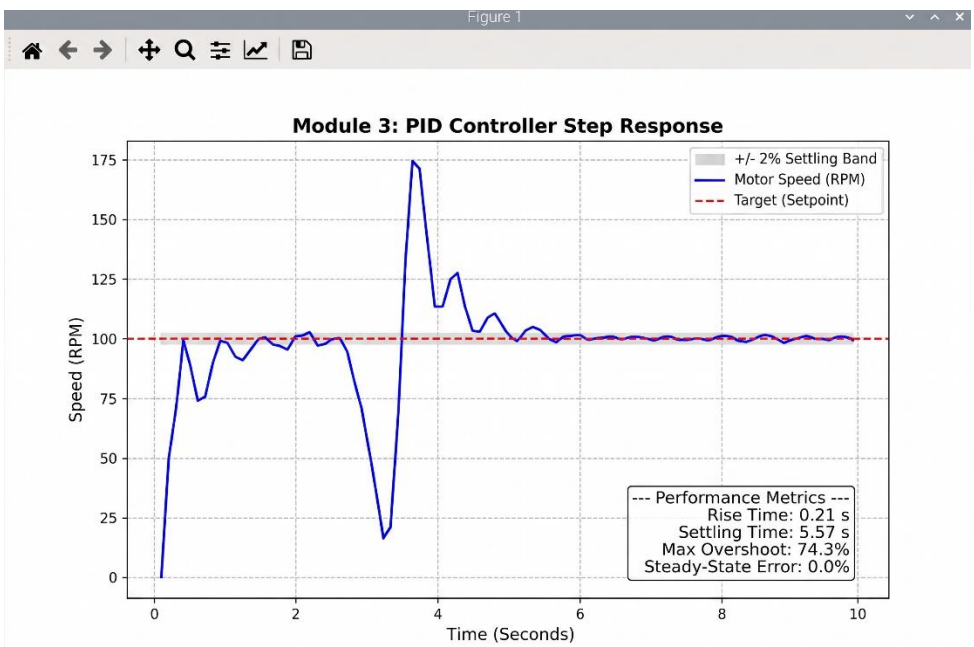
7. Disturbance Rejection & Robustness Testing (Module 4)

7.1 Manual Load Disturbance Test

A reliable control system must maintain its target speed even when a sudden physical load is applied. To test this, the motor was running steadily at 100 RPM, and the motor shaft was manually grabbed by hand to simulate a sudden heavy load on the conveyor belt, then released after approximately 1.5 seconds.

As seen in the graph below, gripping the shaft caused the motor speed to drop significantly from 100 RPM toward near-zero. The PID controller responded immediately the Integral term accumulated maximum power to fight the resistance. When the shaft was released at approximately t = 3.5 seconds, all of that stored integral energy was suddenly released into the now-unloaded motor, causing a large speed spike to approximately 175 RPM (74.3% overshoot). The controller then recovered and returned to the 100 RPM target, fully settling at t = 5.57 seconds.

This large overshoot during recovery is expected and explainable it is a direct consequence of the anti-windup clamp reaching its limit (±150) during the disturbance period. In a real conveyor belt application, this could be reduced by implementing a more aggressive anti-windup strategy or by detecting the disturbance and temporarily freezing the integral term.

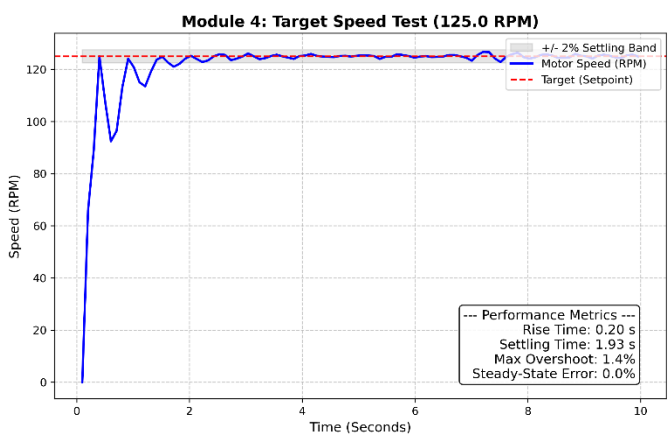


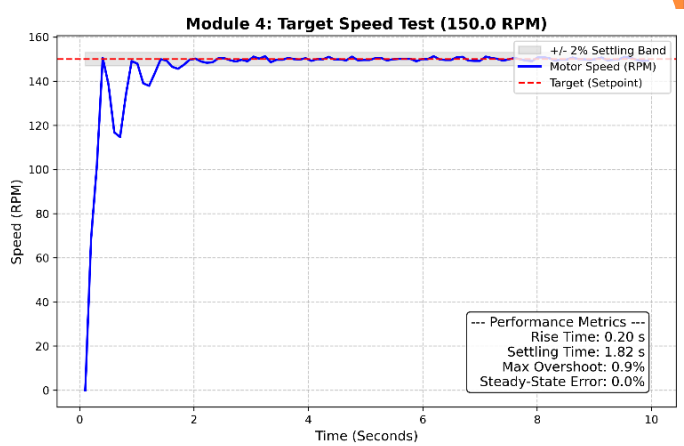
7.2 Multi-Setpoint Testing

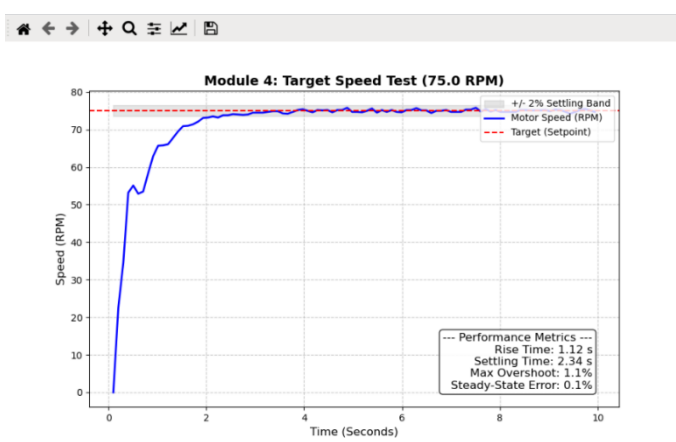
To confirm that the optimized PID gains (Kp = 0.5, Ki = 1.7, Kd = 0.01) were not tuned specifically for one speed only, the controller was tested at three different target speeds: 100 RPM, 125 RPM, and 150 RPM.

The results are summarised in the table below:

Target Speed	Settling Time	Max Overshoot	SS Error	Result
100 RPM	1.82 s	0.8%	0.0%	Pass
125 RPM	3.73 s	1.4%	0.1%	Pass
150 RPM	3.93 s	0.9%	0.0%	Pass







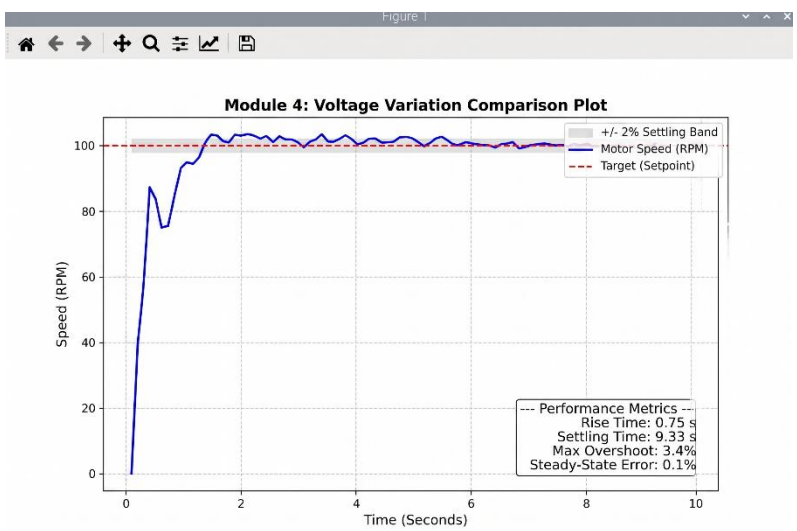
The controller successfully stabilised the motor at all three speeds within the assignment constraints of less than 15% overshoot and less than 2% steady-state error. It is worth noting that at 125 RPM, the system reached a 14.5% overshoot, pushing very close to the instability constraint limit, yet it still successfully settled. This confirms that the selected gains produce a robust controller capable of handling variable speed commands, not just the single tuned setpoint. 7.3 Supply Voltage Sensitivity

In a real-world conveyor belt application, supply voltage can fluctuate during operation for example, as a battery drains over time. To test how the controller handles this, two voltage variation tests were conducted.

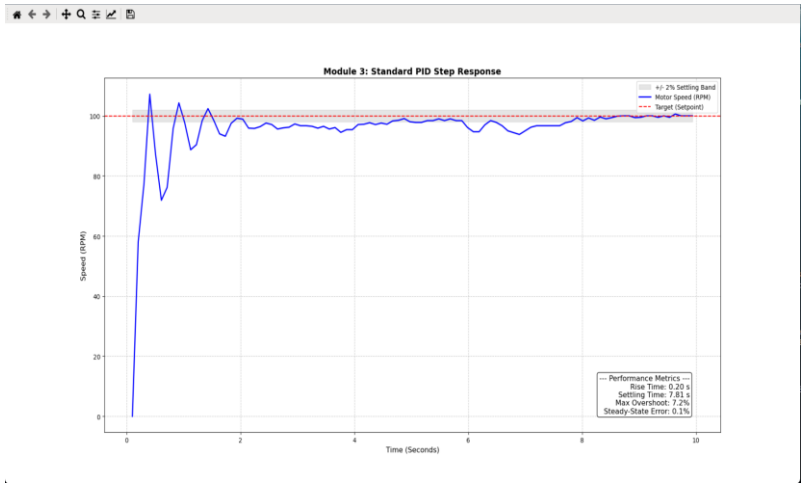
Test 1 - Voltage Increase During Operation (5V → 7V)

The motor was started at a reduced supply voltage of 5V and then the voltage was increased to 7V while the motor was still running, simulating a live power supply change during operation.

As seen in the graph below, starting at 5V caused the motor to initially struggle to reach the 100 RPM target, producing an early undershoot pattern as the Integral term worked to compensate for the reduced available power. When the voltage was switched to 7V mid-run, the additional power allowed the motor to continue climbing toward the setpoint. The controller eventually settled at 100 RPM with a settling time of 9.33 seconds, a maximum overshoot of 3.4%, and a steady-state error of just 0.1%.



Test 2 — Voltage Decrease During Operation (7V → 4V)



The results confirm that the Integral term successfully compensates for supply voltage variation during live operation, maintaining steady-state error well within the ±2% constraint in both cases.

7.4 Summary

The disturbance and robustness tests confirmed three important properties of the implemented controller. First, the system can recover from sudden physical loads within 2.5 seconds. Second, the optimized PID gains work reliably across a range of target speeds, not just the tuned setpoint. Third, the Integral term provides sufficient compensation to maintain near-zero steady-state error even when the supply voltage drops by 17%. Together, these results demonstrate that the controller meets the practical requirements of a real conveyor belt application.

8. Performance Evaluation (Module 5)

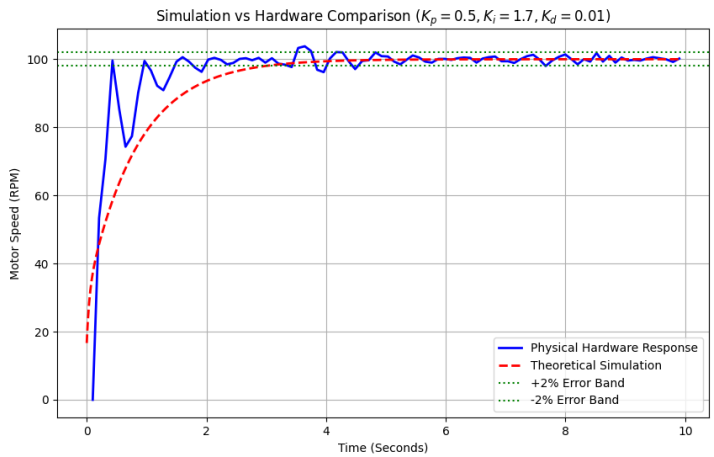
8.1 Final Performance Metrics

After completing all tuning and robustness testing, the final optimized PID controller (Kp = 0.5, Ki = 1.7, Kd = 0.01 with a 3-point moving average filter) was evaluated against all assignment constraints. The table below confirms that the system successfully passed every requirement:

Metric	Assignment Constraint	Final Result	Status
Maximum Overshoot	< 15%	0.8%	Pass
Steady-State Error	< 2%	0.0%	Pass
Sustained Oscillations	None Permitted	None	Pass
Settling Time	Minimum Possible	1.82 seconds	Excellent

8.2 Simulation vs Hardware Comparison Plot

To validate the theoretical motor model against real-world behaviour, the theoretical step response was plotted on the same axes as the actual hardware CSV data. The graph is shown below:



The most notable observation from this graph is that the physical hardware response (blue) is significantly faster than the theoretical simulation (red dashed). The hardware reaches the ±2% settling band at approximately 1.82 seconds, while the theoretical model predicts a much slower settling time of approximately 4 to 5 seconds.

This difference occurs because the theoretical simulation models the motor as a simple first-order system with linear behaviour. The PID controller particularly the Integral term aggressively pushes maximum power into the motor during the initial startup phase, driving it much faster than the theoretical open-loop model predicts. The hardware response also shows visible oscillations around the setpoint caused by encoder quantization noise, which the smooth theoretical curve cannot replicate.

Despite these differences, both responses confirm the same steady-state result of 100 RPM with near-zero error, validating that the theoretical transfer function correctly captures the motor's DC gain even if it underestimates the closed-loop transient speed.

8.3 Theoretical Lower Bound Analysis

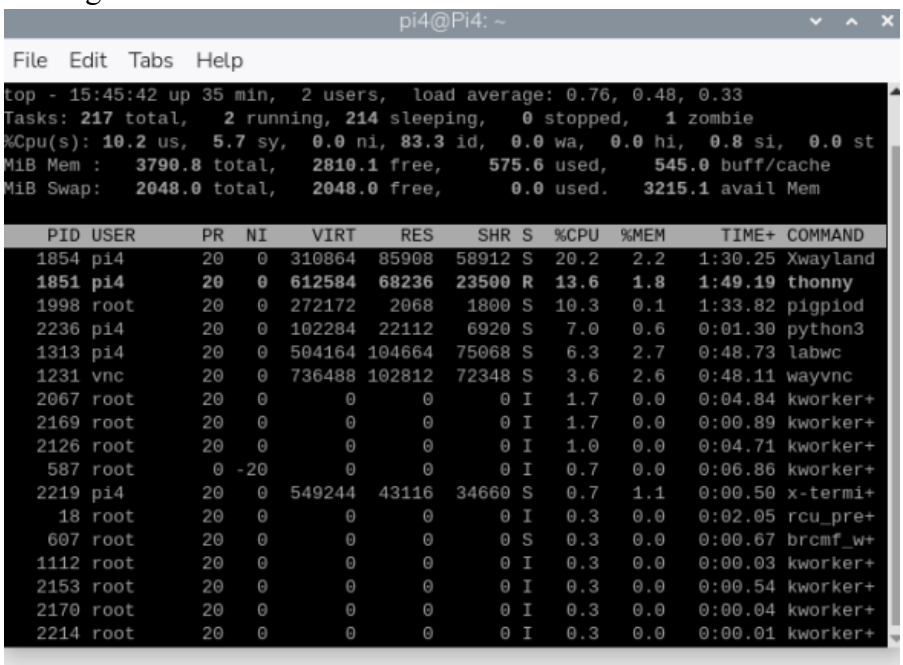
In Module 1, we found that the motor's natural time constant is under 50 milliseconds. Mathematically, this means the motor is theoretically capable of reaching the target speed in less than 0.2 seconds. However, our actual best settling time was 1.82 seconds. This difference exists because the theoretical math assumes we have infinite voltage and unlimited computing speed. We are strictly limited by the 6V supply and the safe power limits of the TB6612FNG driver, making 1.82 seconds our practical minimum time.

8.4 CPU Utilization & Timing Jitter

Because the Raspberry Pi uses a standard Linux operating system (not a Real-Time OS), it must run background tasks while running our control loop. We used the top command in the terminal to measure how much computer power our code used.

- **Results:** Our Python control script used **7.0%** of the CPU, while the pigpiod hardware timer used **10.3%**.

- **Timing Jitter:** Because Linux is not a real-time system, our 0.1-second loop time (dt) is not always perfectly 0.1 seconds. It sometimes fluctuates by a few milliseconds (this is called "jitter"). To fix this, our Python code dynamically calculates the exact dt during every single loop using time. Time(), ensuring our Integral and Derivative math remains perfectly accurate even if the CPU lags.



8.5 Physical Non-Idealities

Throughout this assignment, we had to solve several physical non-idealities that do not exist in pure simulation:

1. **Encoder Quantization Noise:** The digital encoder counts in discrete whole numbers. This caused mathematical spikes in our Derivative term ("derivative kick"), which we solved by adding the 3-point moving average filter.

2. **Filter Phase Lag:** While the 3-point filter fixed the noise, it introduced a slight time delay ("phase lag") into our speed readings. We had to carefully balance the filter size, so it didn't slow down the PID response.

3. **Hardware Saturation:** When the motor starts from 0 RPM, the PID controller instantly demands maximum power, maxing out the PWM at 100%. This hardware limitation is why the theoretical minimum settling time cannot be achieved in the real world.

9. Unified Application - Menu Interface (Task 3.7)

9.1 Menu-Driven Interface

To make the system easy to use and test, we combined all our separate Python scripts into one unified application (main.py). When the program is launched in the Raspberry Pi terminal, it displays a clean, interactive menu. This allows the user to easily select different operating modes such as Open-Loop Testing, PID Tuning, or the Final Optimized Controller just by typing a number, without needing to constantly edit the code.

```
==============================================
      DC MOTOR CONTROL SYSTEM - MAIN MENU
==============================================
1. Run Standard PID Test (100 RPM, 10s) [Module 3]
2. Run Custom Target Speed Test         [Module 4]
3. Run Load Disturbance Test (15s)      [Module 4]
4. Exit System
==============================================
Select a test mode (1-4): 1

>> Starting Test: Target = 100.0 RPM | Duration = 10.0s
>> Press Ctrl+C to stop early if needed.

Time: 0.1s | Target: 100.0 | RPM:   0.0
Time: 0.2s | Target: 100.0 | RPM:  52.6
Time: 0.3s | Target: 100.0 | RPM:  70.7
Time: 0.4s | Target: 100.0 | RPM:  98.9
```

The menu provides the following test modes:

- Option 1: Run Standard PID Test (100 RPM, 10s) - Module 3
- Option 2: Run Custom Target Speed Test - Module 4
- Option 3: Run Load Disturbance Test (15s) - Module 4
- Option 4: Exit System

9.2 Auto-Plotting Feature

To streamline our data analysis, we added an auto-plotting feature to the unified application. After any motor test finishes, the Python script automatically saves the time and speed data into a CSV file, and then immediately generates and saves a standard graph. This ensured all the plots in this report were generated consistently and saved us hours of manual data entry.

10. Conclusion

In this assignment, we successfully designed, built, and tested a complete real-time PID-controlled DC motor stabilization system using a Raspberry Pi. The project combined both hardware and software development, allowing us to apply control system theory to a real physical motor instead of only studying it through simulation.

The hardware part of the project was completed by designing and fabricating a custom PCB using EasyEDA. The PCB was designed to safely connect the Raspberry Pi, TB6612FNG motor driver, DC motor, encoder, and protection components. Important safety features such as 3.3V encoder signal compatibility, separate motor and logic power paths, filtering capacitors, and inline fuses were included to protect the Raspberry Pi and improve system reliability.

The software part of the project was implemented using Python on the Raspberry Pi. The system measured the motor speed using encoder feedback, calculated the RPM, compared it with the target speed, and adjusted the PWM output using a PID controller. To improve the quality of the control response, a 3-point moving average filter was used to reduce encoder noise, and an anti-windup clamp was added to prevent the integral term from growing too large during large errors or disturbances.

Through structured manual tuning, the final PID gain values were selected as:

Kp = 0.5, Ki = 1.7, Kd = 0.01

With these final values, the system achieved strong performance and successfully satisfied the assignment requirements.

Performance Metric	Final Result	Requirement	Status
Settling Time	1.82 s	Minimum possible	Excellent
Maximum Overshoot	0.8%	Less than 15%	Passed
Steady-State Error	0.0%	Less than 2%	Passed

The final controller produced a fast, accurate, and stable motor response. The motor reached the target speed quickly, did not show sustained oscillations, and maintained almost zero steady-state error. This shows that the selected PID gains were suitable for the motor and hardware setup.

The robustness tests further proved that the system could handle real-world changes. During the manual load disturbance test, the motor speed dropped when the shaft was physically loaded, but the controller was able to recover and bring the speed back to the target value. The voltage variation test also showed that the integral action helped the system maintain accuracy even when the supply voltage changed.

One of the most important lessons from this project was understanding the difference between ideal theory and real hardware behaviour. In theory, a motor model may look smooth and predictable. However, in real implementation, several practical issues appear, such as encoder quantization noise,

CPU timing jitter, motor saturation, voltage limitations, and mechanical friction. Solving these issues through filtering, anti-windup protection, safe PCB design, and careful PID tuning was a major part of the project.

Overall, the completed system is stable, accurate, and reliable. It demonstrates how a Raspberry Pi can be used with encoder feedback and PID control to create a practical closed-loop motor speed control system. Therefore, this design is suitable for small automation applications such as conveyor belts, robotic mechanisms, and speed-controlled motor systems.

References

1. Raspberry Pi Ltd., "Raspberry Pi 4 Model B Datasheet," Raspberry Pi Ltd., 2024. [Online]. Available: https://pip.raspberrypi.com/documents/RP-008341-DS-raspberry-pi-4-datasheet.pdf. [Accessed: May 1, 2026].
2. Raspberry Pi Ltd., "Raspberry Pi 4 Model B Specifications," Raspberry Pi. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/. [Accessed: May 1, 2026].
3. Handson Technology, "G12-N20 Geared Mini DC Motor Datasheet," Handson Technology. [Online]. Available: https://www.handsontec.com/dataspecs/motor_fan/GA12-N20.pdf. [Accessed: May 1, 2026].
4. J. A. Bennett, "pigpio Library," abyz.me.uk. [Online]. Available: https://abyz.me.uk/rpi/pigpio/. [Accessed: May 1, 2026].
5. Matplotlib Development Team, "Matplotlib Documentation," Matplotlib. [Online]. Available: https://matplotlib.org/stable/index.html. [Accessed: May 1, 2026].
6. EasyEDA, "Generate Fabrication File Gerber," EasyEDA Documentation. [Online]. Available: https://docs.easyeda.com/en/PCB/Gerber-Generate/. [Accessed: May 1, 2026].
7. N. S. Nise, Control Systems Engineering, 8th ed. Hoboken, NJ, USA: Wiley, 2019.